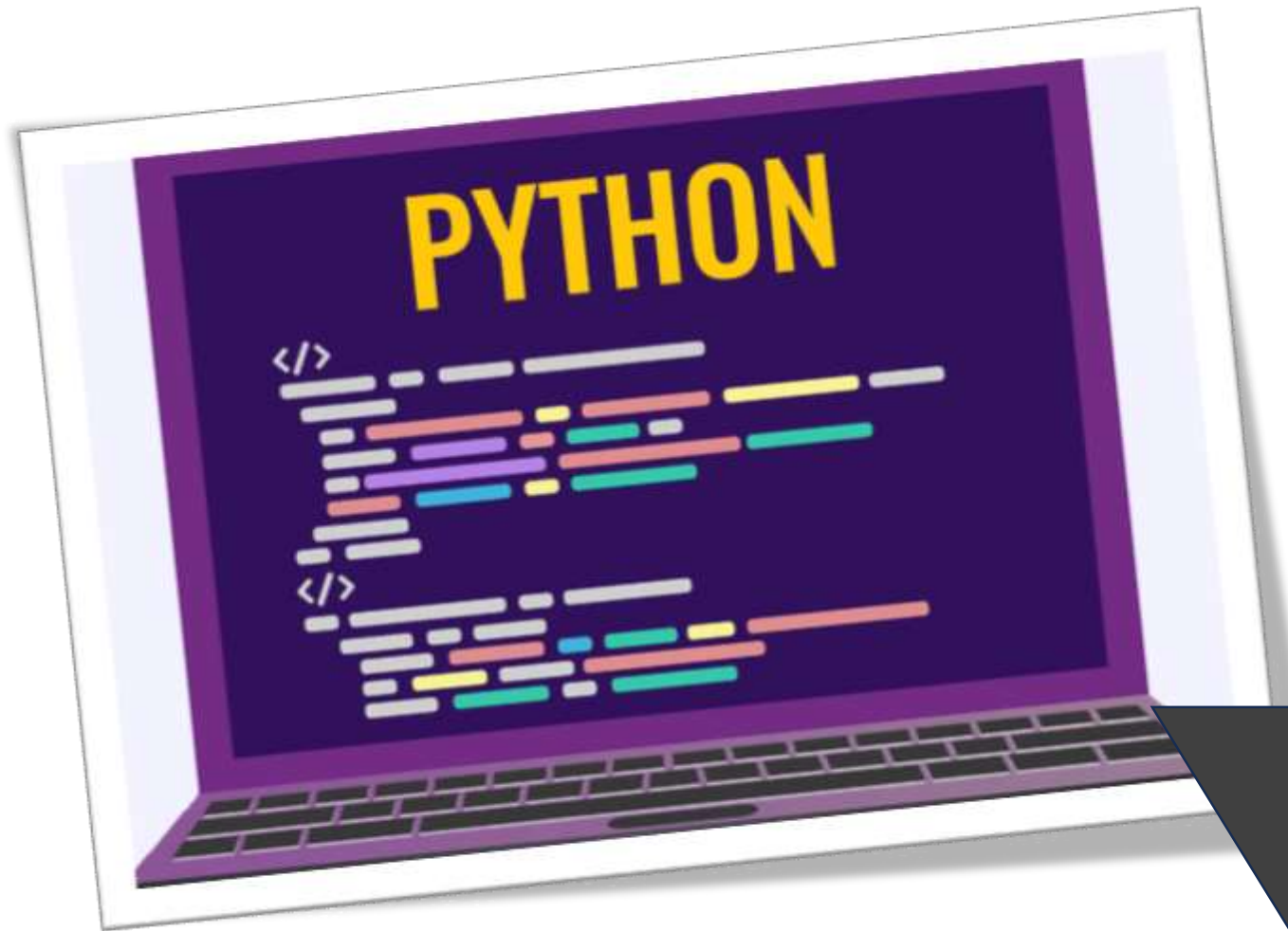




PYTHON VARIABLES AND DATA TYPES

➤ What is a Variable?

✓ Definition:

In Python, a variable is a symbolic name that is a reference or pointer to an object in memory. Once a variable is assigned a value, the variable can be used to refer to that value in the program.

✓ Key Points:

- Python variables are **dynamically typed** — the interpreter infers the data type.
- Variables are just **labels** pointing to memory objects.
- Variables do **not store values** themselves but reference them in memory.

✓ Example:

```
x = 10          # x points to integer object 10  
name = "Ritesh"
```

✦ Internal Concept:

Python stores variables in memory as references to objects:

```
x → 10          (int object in memory)
```



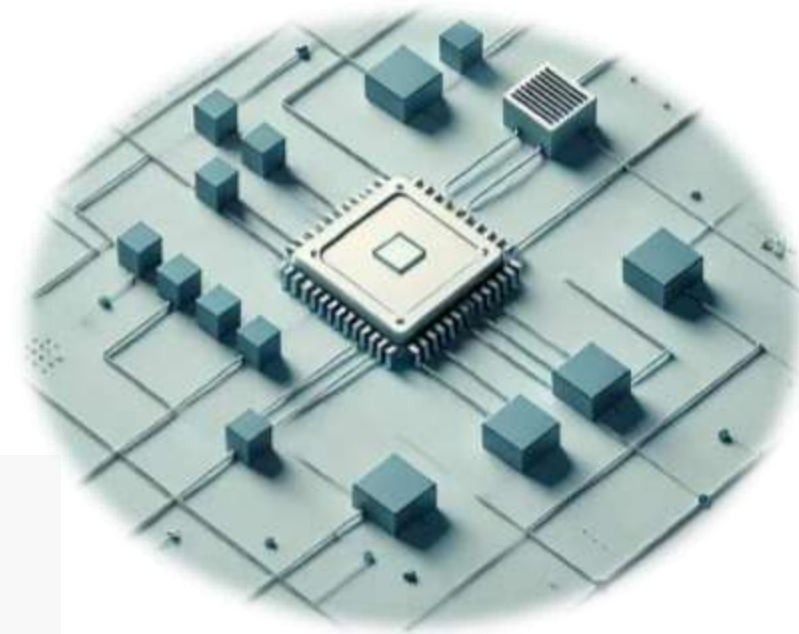
➤ Memory Management & Object Model

◆ Python uses Automatic Memory Management.

- All values are **objects** stored in the heap.
- Python maintains **reference counts** to track object usage.
- Python has a **garbage collector** to clean unused objects.

◆ Everything is an object in Python:

```
x = 5  
print(type(x))  # <class 'int'>
```



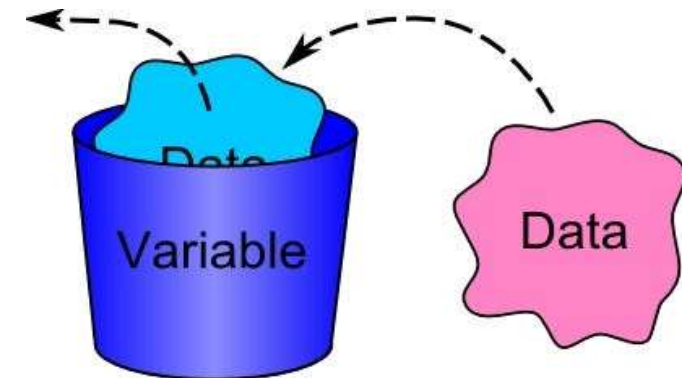
➤ Variable Naming Rules and Conventions

✓ Rules:

- Must begin with a letter (A–Z or a–z) or underscore (_)
- Remaining characters can include letters, numbers, and underscores
- Cannot use reserved **keywords** (for, while, if, etc.)

✓ Convention:

- Use lowercase letters with underscores for readability: user_name, total_sum
- Avoid single-character names like x, y (unless in loops)



➤ Type System in Python – Dynamic & Strong Typing

A **type system** in programming defines how variables are assigned, stored, and used based on their data type (like int, str, etc.).

- Python has a **dynamic** and **strong** type system.



➤ Dynamically Typed Language

In Python, **you don't need to declare the type** of a variable. The **interpreter automatically detects** the type during runtime based on the value assigned.

💡 Example:

```
x = 10                # x is now an integer
print(type(x))        # <class 'int'>

x = "Ten"             # x is now a string
print(type(x))        # <class 'str'>
```

◆ Characteristics:

- Variables are **type-bound at runtime**, not at compile time.
- Variables can **change their data type** during execution.

🔍 Internal Behavior:

Python stores the object in memory and the variable is just a **reference** to it. When the value changes, the reference is redirected to a new object of different type.



➤ Strongly Typed Language

Python is **strongly typed**, which means it does **not implicitly convert** data types in expressions.

Type safety is enforced at runtime.

◆ Why It Matters:

Unlike some languages like JavaScript, Python **raises an error** if you try to combine mismatched types.

💡 Example (Error):

```
x = 10
y = "5"
print(x + y)      # ✗ TypeError: unsupported operand types for +: 'int' and 'str'
```

You must **explicitly convert types**:

```
print(x + int(y)) # ✅ Output: 15
```



Data Types in Python

Python comes with a rich set of built-in data types, categorized as:

Category	Types
Text	str
Numeric	int, float, complex
Sequence	list, tuple, range
Set	set, frozenset
Mapping	dict
Boolean	bool
Binary	bytes, bytearray
None	NoneType



➤ Numeric Data Types

- ◆ **int:** Integer numbers (unlimited precision)
- ◆ **float:** Decimal numbers
- ◆ **complex:** Numbers with real and imaginary parts

```
a = 123
```

```
pi = 3.1415
```

```
z = 2 + 3j
```

🔧 Numeric Built-in Functions:

Function	Purpose	Example
<code>abs(x)</code>	Absolute value	<code>abs(-4) → 4</code>
<code>pow(x,y)</code>	Exponential	<code>pow(2,3) → 8</code>
<code>divmod(a,b)</code>	Quotient and remainder	<code>divmod(10,3) → (3,1)</code>
<code>round()</code>	Round number	<code>round(2.678, 2) → 2.68</code>



➤ String Data Type – str

- A **string** in Python is a **sequence of characters** enclosed in ****single quotes ' '**, **double quotes " "**, or **triple quotes ''' ' / ''' ' '''**.

It is one of the most commonly used data types for storing and manipulating textual data.

✓ Characteristics of Strings:

Characteristic	Description
Ordered	Strings are indexed collections, so each character has a position (index).
Immutable	Once a string is created, it cannot be changed . Any modification creates a new string .
Iterable	You can loop through each character using a for loop.
Slicing	You can extract a part (substring) using string[start:end].
Concatenation	Strings can be combined using + or join().



➤ String Data Type – str

✦✦ Basic String Examples:

```
name = "Ritesh"
print(name[0])      # R      → First character (indexing)
print(name[-1])     # h      → Last character
print(name[1:4])    # ite    → Slicing
print(name + " Kumar") # Ritesh Kumar → Concatenation
```

🔧 Built-in String Functions with Theory and Examples:

Function	Purpose / Theory	Example	Output
len(s)	Returns the length of the string (number of characters).	len("Ritesh")	6
lower()	Converts the string to lowercase .	"Python".lower()	"python"
upper()	Converts the string to uppercase .	"python".upper()	"PYTHON"
replace(a, b)	Replaces all occurrences of substring a with b .	"apple".replace("p", "b")	"abble"



➤ String Data Type – str

Function	Purpose / Theory	Example	Output
split(sep)	Splits the string into a list using the separator (default is space).	"a,b,c".split(",")	["a", "b", "c"]
find(sub)	Returns the first index of the substring, or -1 if not found.	"hello".find("l")	2
isalpha()	Returns True if all characters in the string are alphabets .	"abc".isalpha()	True
isdigit()	Returns True if all characters are digits .	"123".isdigit()	True
startswith(x)	Returns True if the string starts with substring x.	"hello".startswith("he")	True
endswith(x)	Returns True if the string ends with substring x.	"hello".endswith("lo")	True
strip()	Removes leading and trailing whitespaces .	" hi ".strip()	"hi"



➤ String Data Type – str

□ More Practical Code Examples:

```
text = "  Welcome to Python  "

print(text.strip())      # "Welcome to Python"
print(text.lower())      # "  welcome to python  "
print(text.upper())      # "  WELCOME TO PYTHON  "
print(text.find("Python")) # 13
print("abc123".isalnum()) # True (alphabets + numbers)
print("12345".isdigit())  # True
```



➤ List Data Type – list

- A **list** in Python is a **collection of items** that is:
 - **Ordered** (items have a defined index)
 - **Mutable** (can be changed after creation)
 - **Allows duplicates**
 - Can store **heterogeneous data** types (e.g., integers, strings, floats, even other lists)

Lists are created using square brackets [].

```
fruits = ["apple", "banana", "cherry"]  
numbers = [1, 2, 3, 4]  
mixed = [1, "hello", 3.14, True]
```



List Data Type – list

✓ Characteristics of Lists:

Characteristic	Description
Mutable	You can add, remove, or update elements after the list is created.
Ordered	Elements are stored in a defined sequence and can be accessed via index.
Allows Duplicates	Duplicate values are allowed.
Heterogeneous Data	A list can contain different data types.
Nested Lists	Lists can contain other lists as elements.

✓ Examples:

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[0])           # apple  
print(fruits[-1])          # cherry  
print(len(fruits))          # 3  
print(fruits[1:])           # ['banana', 'cherry']
```



➤ List Data Type – list

🔧 Common List Methods with Theory and Examples:

Method	Description	Example	Output
append(x)	Adds element x to the end of the list	fruits.append("mango")	["apple", "banana", "cherry", "mango"]
insert(i, x)	Inserts element x at index i	fruits.insert(1, "orange")	["apple", "orange", "banana", "cherry"]
pop()	Removes and returns the last element	fruits.pop()	"cherry" (and list is updated)
remove(x)	Removes the first occurrence of element x	fruits.remove("banana")	List without "banana"
sort()	Sorts the list in place (ascending by default)	numbers.sort()	[1, 2, 3, 4]
reverse()	Reverses the order of elements in the list	fruits.reverse()	List in reverse order



➤ List Data Type – list

□ More Practical Code Examples:

```
numbers = [3, 1, 4, 2]
numbers.sort()           # [1, 2, 3, 4]

fruits = ["apple", "banana", "cherry"]
fruits.append("mango")   # ['apple', 'banana', 'cherry', 'mango']
fruits.pop()             # Removes 'mango'
fruits.insert(1, "kiwi") # Insert 'kiwi' at index 1
fruits.remove("banana")  # Remove 'banana'
```



➤ Tuple Data Type – tuple

- A **tuple** in Python is an **ordered, immutable collection** of items.

It is similar to a list, but unlike lists, **tuples cannot be modified** (i.e., no item can be added, removed, or changed once created).

Tuples are created using **parentheses ()**.

```
coordinates = (10, 20)
info = ("Ritesh", 25, "Male")
```

✓ Characteristics of Tuples:

Characteristic	Description
Immutable	Cannot change elements after creation — prevents accidental modifications.
Ordered	Elements are stored in a defined order and can be accessed via index.
Allows Duplicates	Can contain repeated elements.
Heterogeneous	Can store data of different types (int, str, float, etc.).
Faster than Lists	Due to immutability, they are more memory-efficient and faster in lookup.
Hashable	Can be used as keys in dictionaries (if all elements are hashable).



➤ Tuple Data Type – tuple

✓ Examples:

```
coordinates = (10, 20)
print(coordinates[0])      # 10
print(coordinates[-1])     # 20

person = ("Alice", 30, "Engineer")
print(person[1])           # 30

nested = (1, [2, 3], 4)
# nested[1][0] = 100  # Allowed (list inside tuple is mutable)
```

🔧 When to Use Tuples Instead of Lists:

- When your data should **not change** (like coordinates, RGB color codes, etc.).
- When performance (speed/memory) is critical.
- When using as **dictionary keys** or elements of a set.

🔧 Tuple Methods with Theory and Examples:

Method	Description	Example	Output
count(x)	Returns the number of times element x appears	(1, 2, 2, 3).count(2)	2
index(x)	Returns the first index where element x is found	(10, 20, 30).index(20)	1



➤ Set Data Type – set

- A **set** in Python is an **unordered** collection of **unique** and **mutable** elements.

Sets are useful when you want to store multiple items but **automatically eliminate duplicates**, and perform operations like **union, intersection, and difference**.

- Created using **curly braces {}** or the **set()** constructor.

```
s = {1, 2, 3}
t = set([3, 4, 5])
```

✓ Characteristics of Sets:

Characteristic	Description
Unordered	Elements have no index or order. You can't rely on order while accessing.
Mutable	You can add or remove elements after creation.
No Duplicates	Duplicate values are automatically removed .
Heterogeneous	Can store different types (int, str, float, etc.), but elements must be hashable .
Optimized	Very fast for membership testing (x in set) and mathematical set operations.



➤ Set Data Type – set

✓ Example:

```
s = {1, 2, 2, 3}
print(s)           # Output: {1, 2, 3} → Duplicates removed automatically
```

🔧 Common Set Methods with Theory and Examples:

Method	Description	Example	Output
add(x)	Adds element x to the set (if not already present).	s.add(4)	{1, 2, 3, 4}
remove(x)	Removes element x. Raises error if not found.	s.remove(2)	{1, 3}
discard(x)	Removes element x. No error if not found.	s.discard(5)	Safe execution
union(set2)	Returns a new set with elements from both sets.	a.union(b)	{1, 2, 3, 4, 5}
intersection(set2)	Returns a set of elements common to both sets.	a.intersection(b)	{3}
difference(set2)	Returns elements in a not in b.	a.difference(b)	{1, 2}



➤ Set Data Type – set

□ Code Example:

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a.union(b))      # {1, 2, 3, 4, 5}
print(a.intersection(b)) # {3}
print(a.difference(b))  # {1, 2}

a.add(6)                # {1, 2, 3, 6}
a.discard(10)           # No error even if 10 is not in set
```

! Notes:

- Sets do **not support indexing**: `s[0]` will raise `TypeError`.
- Cannot store **unhashable types** like other sets or lists.

```
invalid_set = {1, 2, [3, 4]} # ❌ Error: unhashable type: 'list'
```



Dictionary Data Type – dict

- A **dictionary** in Python is a **collection of key-value pairs** where:
 - Each **key is unique and immutable**.
 - Each **value can be of any data type**.
 - Dictionaries are **unordered** (in versions < 3.7) but maintain insertion order in Python 3.7+.
 - Created using curly braces {} or the dict() constructor.

```
student = {  
    "name": "Ritesh",  
    "age": 22  
}
```



➤ Dictionary Data Type – dict

✓ Characteristics of Dictionaries:

Characteristic	Description
Key-Value Storage	Data is stored as key: value pairs.
Mutable	You can add, change, or remove key-value pairs after creation.
Unordered	Items have no index (although order is preserved from Python 3.7+).
Unique Keys	Keys must be unique; if repeated, the last one overwrites the earlier.
Immutable Keys Only	Keys must be immutable types like strings, numbers, or tuples.



➤ Dictionary Data Type – dict

✓ Example:

```
student = {  
    "name": "Ritesh",  
    "age": 22,  
    "course": "Python"  
}  
  
print(student["name"])    # Ritesh  
print(student.get("age")) # 22
```

!Notes:

- Keys must be immutable: strings, numbers, or tuples.
- Values can be mutable or immutable — lists, strings, numbers, other dictionaries, etc.
- Nested dictionaries are supported:

```
person = {  
    "name": "Ritesh",  
    "address": {  
        "city": "Delhi",  
        "pincode": 110001  
    }  
}
```



➤ Dictionary Data Type – dict

🔧 Common Dictionary Methods with Theory and Examples:

Method	Description	Example	Output
get(k)	Returns the value for key k. If not found, returns None (no error).	student.get("age")	22
keys()	Returns a view of all keys in the dictionary.	student.keys()	dict_keys(['name', 'age'])
values()	Returns a view of all values in the dictionary.	student.values()	dict_values(['Ritesh', 22])
items()	Returns a view of all key-value pairs as tuples.	student.items()	dict_items([('name', 'Ritesh')])
update({...})	Adds new or updates existing key-value pairs.	student.update({"age": 23})	Updates 'age' to 23
pop(k)	Removes the key k and returns its value. Raises error if key is missing.	student.pop("age")	Removes 'age' and returns 22



➤ Dictionary Data Type – dict

```
student = {  
    "name": "Ritesh",  
    "age": 22,  
    "course": "Python"  
}  
  
# Accessing values  
print(student["name"])          # Ritesh  
  
# Using get() safely  
print(student.get("email"))     # None  
  
# Updating dictionary  
student["age"] = 23  
student.update({"email": "ritesh@example.com"})  
  
# Removing a key  
student.pop("course")  
  
# Viewing keys and values  
print(student.keys())           # dict_keys(['name', 'age', 'email'])  
print(student.values())         # dict_values(['Ritesh', 23, 'ritesh@example.com'])
```



➤ Boolean Type – bool

□ The bool data type in Python represents **Boolean values**:

✓ True and ✗ False

Booleans are commonly used in:

- **Conditional statements**
- **Logical operations**
- **Flags and control flow**

✓ **Example:**

```
is_active = True
print(is_active + 5)    # Output: 6 → True is treated as 1
```

✓ **Key Characteristics:**

Property	Description
Two Values Only	True and False (case-sensitive)
Subclass of int	Internally, True = 1 and False = 0
Used in Logic	Essential for if, while, comparisons, and logical operators (and, or, not)
Returned by Tests	Functions like isalpha(), isdigit(), comparisons, etc., return bool



➤ Boolean Type – bool

✦ Boolean in Expressions:

```
print(10 > 5)           # True
print(3 == 7)           # False
print(bool(0))           # False
print(bool("Hello"))     # True
print(bool(""))          # False
```

🔄 Common Use Cases:

Use Case	Example	Result
Comparison	5 < 10	True
Empty Check	bool([])	False
Logical Operation	True and False	False
Condition in if statement	if is_active:	Executes if True
Flags and Status Indicators	is_logged_in = False	Track state



➤ None Type – NoneType

✓ Represents:

- Absence of a value or null
- Used as default value in functions or variable initialization

```
x = None
```

➤ Type Checking using type()

```
x = [1, 2, 3]  
print(type(x))  # <class 'list'>
```



Type Casting and Conversion

✓ Purpose:

Convert data from one type to another.

✓ Common Functions:

Function	Converts To	Example
int()	Integer	int("10") → 10
float()	Float	float("5.5") → 5.5
str()	String	str(100) → '100'
list()	List	list("abc") → ['a','b','c']
tuple()	Tuple	tuple([1,2]) → (1,2)



➤ Mutability Table

Data Type	Ordered	Mutable	Example
int	×	×	10
float	×	×	3.14
str	✓	×	"Hello"
list	✓	✓	[1,2,3]
tuple	✓	×	(1,2,3)
set	×	✓	{1,2,3}
dict	×	✓	{"a":1}



Practical Use Cases of Data Types

Data Type	Real-world Use
list	Store dynamic student names
tuple	Coordinates (x, y), dates
set	Unique product IDs
dict	Student records, API responses
str	Messaging, display text
bool	Conditions, state flags





